ADA 26. oktober 2026

Shellsort, rekurrens og Quicksort

Porteføljeopgave

- Findes under *Ressourcer* på *itslearning.*
- Løses individuelt.
- Godkendelse er eksamensforudsætning.
- Afleveres senest 16. november.

Shellsort

Shellsort er en 'multi-pass' algoritme. Hvert pass er en insertion sort af sekvenser bestående af hvert *h.* element for et for et fast (fixed) mellemrum *h* (også kaldt inkrementet). Dette kaldes *h*-sortering.

Analysen er besværlig. Simuleringer antyder best-case på $O(N^{5/4})$; men det kan ikke bevises.

Worst case er $O(N^2)$.

Shellsort - eksempel

Orig.	**81**	**94**	**11**	**96**	**12**	**35**	**17**	**95**	**28**	**58**	**41**	**75**	**15**
5	35	17	11	28	12	41	75	15	96	58	81	94	95
3	28	12	11	35	15	41	58	17	94	75	81	96	95
1	11	12	15	17	28	35	41	58	75	81	94	95	96

	0	1	2	3	4	5	6	7	8	9	10	11	12
0, 5, 10		1, 6, 11			2, 7, 12				3, 8			4, 9	
81,35,41		94,17,75			11,95,15				96,28			12,58	

Shellsort - eksperimenter

N = 1024

Shellsort	58.893	$N^{1,58}$
Bubblesort	261.563	$N^2/4$
1024^2	1.048.56	

Shellsort

- Som eksemplet illustrerer, er de subarrays, som Shellsort opererer på små til at starte med; senere bliver de større, men næsten sorterede. I begge til fælde er insertion sort effektiv.
- Shellsort is ustabil: det kan forekomme, at den ændrer rækkefølgen af elementer med samme værdi.
- Den har en 'naturlig' adfærd (lige som bubblesort), eftersom den afvikles hurtigere, hvis tabellen er relativt sorteret (få inversions).

Konklusion

- Inversions
- $O(N^2)$ – bubble, insertion, selection
- O(N log N) – heap, quick, merge
- O(N) - bucket
- Vælg den mest hensigtsmæssige på baggrund af det talmateriale, der skal sorteres.
- Problemet er næppe vigtigt, hvis der er færre end 100.000 ikke alt for store objekter, og sorteringen derfor kan udføres i memory.

Hvad er rekurrens?

- En rekurrens (recurrence) er en ligning, der beskriver en funktion ud fra dens værdi på mindre input.
- Eksempel: $T(N) = 2T(N/2)+N$
- Bruges især til at analysere rekursive algoritmer.
- Viser, hvordan problemet nedbrydes i delproblemer.

Eksempel på rekurrens

- Eksempel fra en algoritme, der opdeler problemet i to halvdele og samler resultaterne på lineær tid:
- $T(N) = 2T(N/2) + N$
- To rekursive kald på størrelse N/2
- Plus lineært arbejde for at kombinere resultaterne.

Hvorfor er rekurrenser vigtige?

- Bruges til at forudsige køretiden for rekursive algoritmer.
- Hjælper med at forstå væksten i beregningstid, når input vokser.
- Typiske anvendelser:
- Merge Sort: $T(N) = 2T(N/2) + O(N)$
- Binær søgning: $T(N) = T(N/2) + O(1)$
- Tower of Hanoi: $T(N) = 2T(N-1) + O(1)$

Intuitionen bag rekurrens

- En rekurrens beskriver selv-lighed i beregningen.
- Hvert kald deler problemet i mindre versioner af sig selv.
- Når basis-tilfældet nås, stopper rekursionen.
- Summen af arbejdet i alle niveauer giver den samlede kompleksitet.

Teleskop-kikkert



Vi kan trække den ud og vi kan folde den sammen.

Telescoping

- På dansk *teleskopering* 😉
- “En teleskoperende sum er en sum, hvor de fleste led går ud med hinanden.”
- Eller: “at forkorte en sum ved, at led går ud med hinanden”

Telescoping som bevismetode

- Først opstiller vi rekurrens-ligningen.
- Fx: $T(N) = 2T(N/2) + N$ (merge-sort)
- Vi stater med at dividere med N på begge sider.
- Herefter opstiller vi en ny ligning, hvor N erstattes med N/2.
- Hvis vi fortsætter med at lave nye ligninger, hvor N erstattes med N/2, får vi ligninger, hvor første udtryk på højresiden er lig venstresiden i næste ligning.
- Når vi kommer til bunds, dvs. til base-case, lægger vi alle venstresider og alle højresider sammen.
- Herefter går ovennævnte udtryk ud med hinanden.
- Til sidst har vi derfor kun venstresiden af første ligning tilbage, og den er lig med højresiden af sidste lignings første udtryk plus summen af højresidernes sidste udtryk.
- Tidskompleksiteten kan nu beregnes ved at gange igennem med N.

Analyse af MergeSort

$T(N) = 2T(N/2) + N$

$T(N)/N = T(N/2)/(N/2) + 1$

$T(N/2)/(N/2) = T(N/4)/(N/4) + 1$

$T(N/4)/(N/4) = T(N/8)/(N/8) + 1$

….. fortsæt til base-case.

$T(2)/2 = T(1)/1 + 1$

..... læg højre- og venstresider sammen og reducér.

$T(N)/N = T(1)/1 + \log N$

$T(N) = N + N \log N \rightarrow O(N \log N)$

Teleskop-kikkert



Vi kan trække den ud og vi kan folde den sammen.

Om QuickSort

- *QuickSort* er meget effektiv ved sortering af store talmængder i memory.
- Man starter med at finde et godt bud på talmængdens 'midterste' værdi (*pivot*), fx . *median3.*
- Pegepinde placeres i hver sin ende af tabellen.
- Elementer swappes, hvis de står forkert i forhold til pivotet.
- Der fortsættes indtil pegepindene mødes.
- Denne proces kaldes *partitionering.*
- Herefter foretages rekursive kald på de nye tabeller.
- Når tabellerne bliver små, fx 20 elementer, færdiggøres sorteringen med *InsertionSort*.

Quicksort

```
1   public void quickSort(int array[], int left, int right)
2   {
3           if (left + CUTOFF < right)
4           {
5                   int pivot = median3(array, left, right);
6                   int i = left, j = right -1;
7                   for ( ; ; )
8                   {
9                           while( array[i] < pivot) i++;
10                          while( array[j] > pivot) j--;
11                          If (i < j)
12                                  swap(array, i, j);
13                          else
14                                  break;
15                  }
16                  swap(array, i, right - 1);
17
18                  quickSort(array, left, i - 1);
19                  quickSort(array, i + 1, right);
20          }
21          else
22                  insertionSort(array, left, right);
23  }
```

Mere om QuickSort

- Eftersom pivotet sjældent rammer det midterste element, er der ikke tale om halvering af problemet.
- Når algoritmen alligevel er meget effektiv, skyldes det partitioneringsprocessens effektivitet.
- Et enkelt swap kan eliminere titusindvis af inversions.

Analyse af worst case QuickSort

T(N) = T(N-1) + cN

T(N-1) = T(N-2) + c(N-1)

T(N-2) = T(N-3) + c(N-2)

T(N-3) = T(N-4) + c(N-3)

…. fortsæt til base case

T(2) = T(1) + c(2)

…… læg højre- og venstresider sammen og reducér.

T(N) = T(1) + c$\sum_{i=2}^{N} i$ -> $\theta(N^2)$